Software Quality Assurance Metrics

Exam-Oriented Question Bank & Complete Answer Key

PART 1: QUESTION PAPER OVERVIEW

10-Mark Questions (Long Answer)

1. Define Software Quality Metrics and discuss in detail the three primary categories of software metrics: Product Metrics, Process Metrics, and Project Metrics, including their specific metrics, formulas, and significance in Software Quality Assurance.
2. Explain the concept of Software Quality according to the ISO 9126 standard. Detail all six main quality characteristics and discuss their role in establishing software quality benchmarks.
3. What is Total Quality Management (TQM)? Discuss the six core principles of TQM and explain how TQM transforms organizational culture and practices in the software industry.
4. Define Defect Prevention (DP). Explain the major defect prevention techniques including Root Cause Analysis (RCA) and contrast Defect Prevention with Defect Detection in detail using a comparison table.
5. Describe the complete step-by-step methodology of Software Quality Metrics Analysis and explain how statistical and graphical tools (Control Charts, Pareto Charts, Scatter Diagrams, and Trend Lines) drive continuous quality improvement.

5-Mark Questions (Medium Answer)

6. Explain the PDCA Cycle (Plan-Do-Check-Act / Deming Cycle) as a continuous quality improvement model in software process management.
7. What is Benchmarking in SQA? Explain its core objective and describe the three major types of benchmarking used in software development.
8. Explain Pareto Analysis (the 80/20 principle) and describe its practical application in software defect management and prioritization.
9. Describe Control Charts in software quality control, detailing the Central Line (CL), Upper Control Limit (UCL), Lower Control Limit (LCL), and process variations.
10. Explain the Cause-and-Effect (Ishikawa/Fishbone) Diagram and detail the five typical cause categories used in software defect analysis.
11. Explain Cyclomatic Complexity V(G) and Mean Time to Failure (MTTF) as product quality metrics, providing their mathematical formulas and structural components.
12. Explain Defect Removal Efficiency (DRE) and Rework Percentage as process quality metrics, detailing their mathematical formulas and operational significance.
13. Compare Product Metrics, Process Metrics, and Project Metrics using a structured comparison table across key evaluation dimensions.

2-Mark Questions (Short Answer)

14. Define a Software Quality Metric and state its primary role in decision-making.
15. Define Defect Density (DD) and write its mathematical formula.
16. Explain Mean Time to Repair (MTTR) in software maintainability.
17. Define Inspection Yield and state its purpose in software review processes.
18. Define Cost Performance Index (CPI) and Effort Variance in project metrics.
19. Explain the concept of Continuous Improvement (Kaizen) in TQM.
20. Briefly describe the 5 Whys Technique used in Root Cause Analysis.
21. What is a Scatter Diagram and how is it used in software metrics analysis?
22. If the defect density of a software system drops from 1.5 defects/KLOC in Version 1 to 0.9 defects/KLOC in Version 2, calculate the percentage quality improvement.

PART 2: COMPLETE ANSWER KEY

10-MARK QUESTIONS & ANSWERS

Q1. Define Software Quality Metrics and discuss in detail the three primary categories of software metrics: Product Metrics, Process Metrics, and Project Metrics, including their specific metrics, formulas, and significance in Software Quality Assurance. [10 Marks]

Definition & Overview

Software Quality Metrics are quantitative indicators that measure attributes of software products, development processes and management performance. They give an objective numerical basis for decisions on reliability, process control and whether quality expectations are being met.

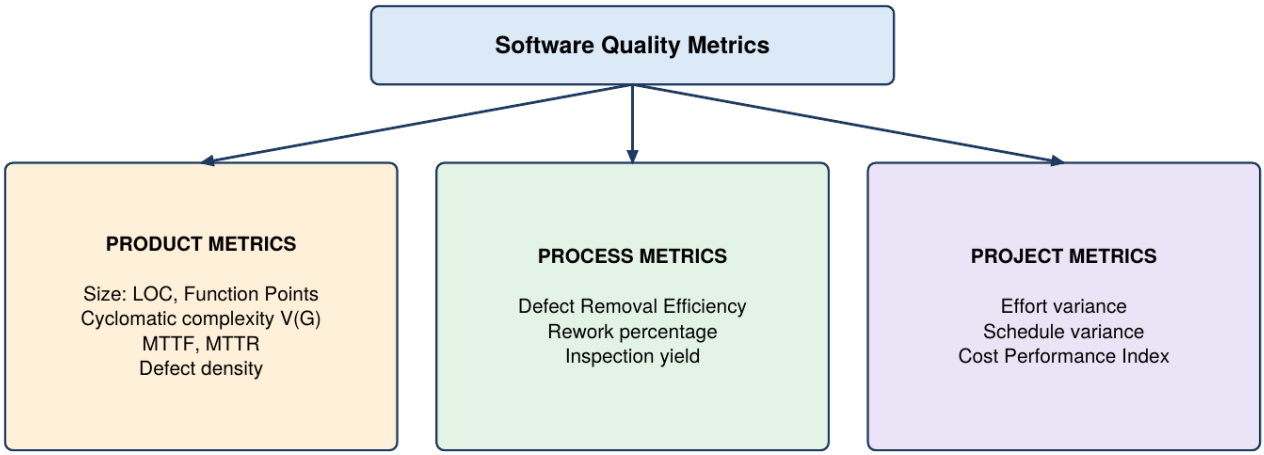


Fig 1: Classification of software quality metrics

- **Definition of Software Metrics:** Quantitative measurements of software attributes that evaluate efficiency, quality and performance. They turn subjective quality goals into measurable targets and give feedback across all SDLC phases.
- **Product Metrics Overview:** Measure the software product itself: its size, internal complexity and operational reliability, across code, design artefacts and documentation. They help find high-risk modules that need refactoring or extra testing.
- **Product Metric: Size (LOC and FP):** Lines of Code (LOC) count physical or logical source lines; Function Points (FP) measure the functionality delivered to the user. Size is the denominator for defect density and productivity.
- **Product Metric: Cyclomatic Complexity V(G):** Counts the linearly independent paths in the control flow graph: `V(G) = E - N + 2P` (E = edges, N = nodes, P = connected components). High values mean intricate logic, higher defect risk and more testing effort.
- **Product Metric: Reliability (MTTF and MTTR):** `MTTF = Total operational time / Number of failures` gives the average run time before a failure. MTTR is the average effort to fix a defect, and both should improve across builds.
- **Process Metrics Overview:** Quantify how efficiently and effectively development and maintenance processes work, using defect discovery rates, review thoroughness and rework. Better process metrics cut cost and time-to-market.
- **Process Metric: Defect Removal Efficiency (DRE):** Percentage of total defects found and fixed before release: `DRE = (Defects found before release / Total defects) x 100`. A high DRE shows effective internal verification and inspection.
- **Process Metrics: Rework Percentage and Inspection Yield:** Rework percentage is the share of effort spent correcting defects instead of building features. Inspection yield shows how well reviews catch defects early, and lower rework saves resources.
- **Project Metrics Overview:** Assess project management performance: budget consumption, schedule adherence, resource productivity and progress, so deliverables stay on time and on budget.
- **Project Metrics: Variances and CPI:** Effort Variance compares planned and actual effort; Schedule Variance shows delays or gains. `CPI = Earned Value / Actual Cost` measures cost efficiency and warns of project slippage early.

Q2. Explain the concept of Software Quality according to the ISO 9126 standard. Detail all six main quality characteristics and discuss their role in establishing software quality benchmarks. [10 Marks]

Definition & Overview

Software Quality is the degree to which software meets specified requirements and satisfies customer expectations. ISO 9126 defines quality through six main characteristics covering both internal code properties and external behaviour.

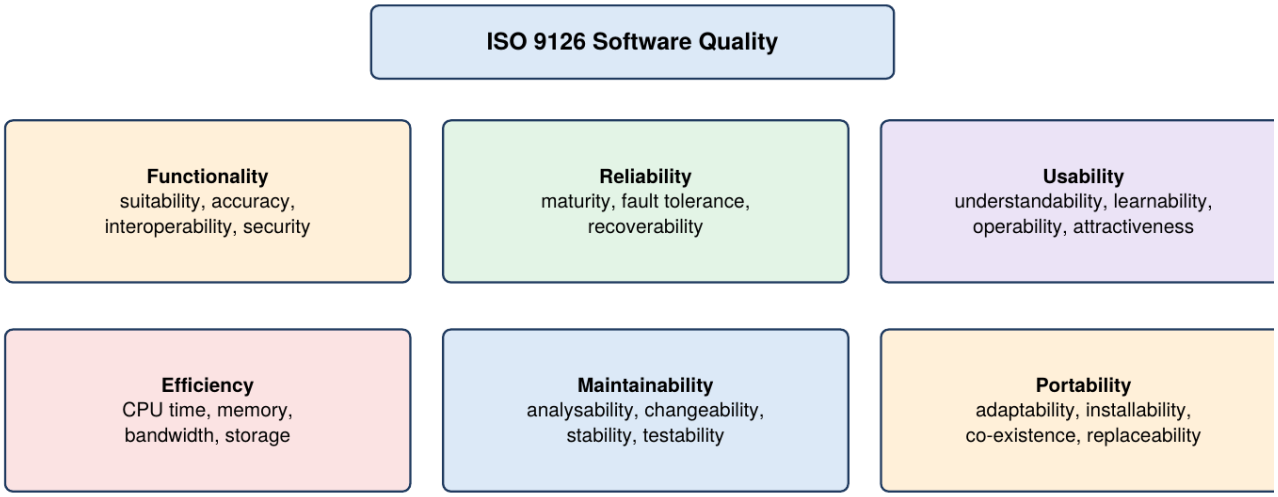


Fig 2: ISO 9126 quality characteristics and sub-attributes

- **ISO 9126 Quality Framework:** A structured model to define, categorise and evaluate product quality, used as the baseline for QA benchmarks and verification criteria.
- **Functionality:** Ability to provide functions that meet stated and implied needs, correctly and completely. Sub-attributes: suitability, accuracy, interoperability, security, compliance.
- **Reliability:** Ability to maintain a specified performance level over time without unexpected crashes. Sub-attributes: maturity, fault tolerance, recoverability, compliance.
- **Usability:** Effort users need to learn, operate and understand the software, covering interface clarity and satisfaction. Sub-attributes: understandability, learnability, operability, attractiveness.
- **Efficiency:** Performance relative to resources used: CPU time, memory, bandwidth and storage. Higher efficiency gives fast response with low resource use.
- **Maintainability:** Ease of modifying, correcting, improving or adapting the software; it drives post-release cost. Sub-attributes: analysability, changeability, stability, testability.
- **Portability:** Ease of moving the software to another hardware, software or operating environment. Sub-attributes: adaptability, installability, co-existence, replaceability.
- **Role in Benchmark Establishment:** SQA teams attach measurable targets to each characteristic during requirements (e.g., MTTF for reliability, execution time for efficiency). These benchmarks guide inspections, testing and release decisions.
- **Impact on Defect Reduction:** Non-functional qualities such as maintainability and portability get equal attention to functionality. Finding usability or efficiency weaknesses early avoids costly re-engineering and lowers post-release defects.
- **Alignment with TQM:** Puts user satisfaction at the centre and promotes holistic measurement instead of post-development bug fixing, so products meet international standards.

Q3. What is Total Quality Management (TQM)? Discuss the six core principles of TQM and explain how TQM transforms organizational culture and practices in the software industry. [10 Marks]

Definition & Overview

Total Quality Management (TQM) is a management philosophy of continuous process improvement, customer satisfaction and involvement of every employee. In software it moves the culture from reactive defect detection to proactive quality prevention.

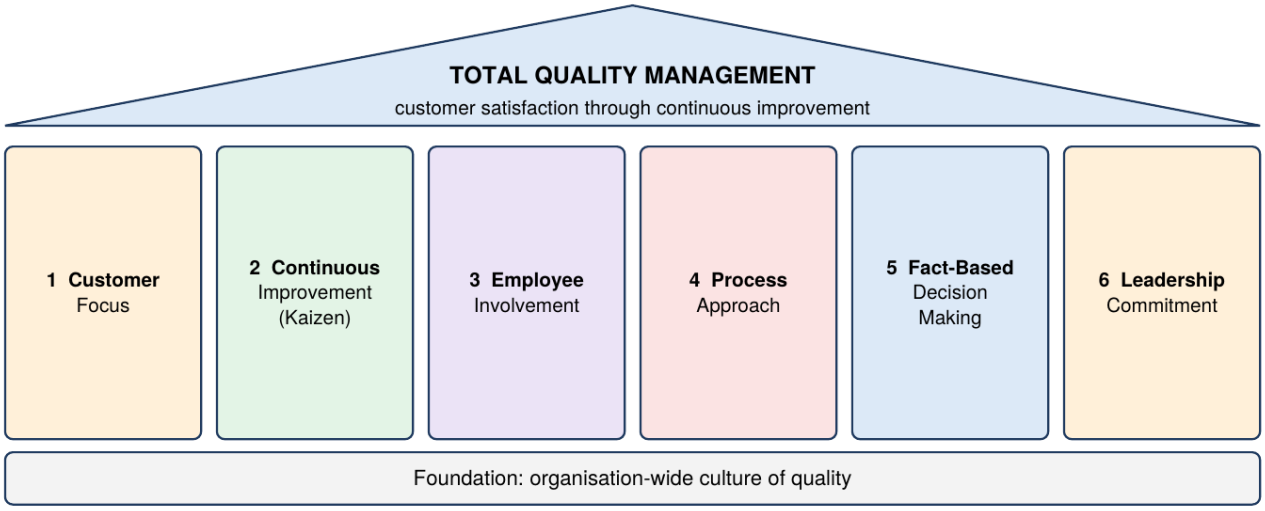


Fig 3: Six principles of TQM

- **Definition and Philosophy:** An organisation-wide approach that keeps refining processes, products and services for lasting customer satisfaction. Quality is built into every SDLC phase and shared by everyone, not left to the testing team.
- **Principle 1: Customer Focus:** Quality is defined by user needs and satisfaction, so specifications must match user requirements and usability goals. Features that miss user needs are a quality failure.
- **Principle 2: Continuous Improvement (Kaizen):** Process management relies on small, steady improvements. Teams analyse process data routinely, and gains in coding standards and review yield compound over time.
- **Principle 3: Employee Involvement:** Everyone from architects to testers owns quality. Employees are empowered to report bottlenecks and suggest fixes, creating shared ownership.
- **Principle 4: Process Approach:** Development is a network of linked processes (requirements, design, coding, testing, release). Analysing inputs and outputs pinpoints defect-causing phases and reduces variability.
- **Principle 5: Fact-Based Decision Making:** Decisions rest on data and metrics such as defect density, review efficiency and control charts, removing guesswork and bias.
- **Principle 6: Leadership Commitment:** Management funds training, tools and infrastructure, champions quality goals and links TQM to business strategy.
- **Transformation of Mindset:** Shifts teams from testing for defects after coding to building quality into the process, replacing costly post-release fixes with prevention.
- **Standardisation and Automation:** Leads to coding guidelines, rigorous peer reviews and automated static analysis, which cut human error and complexity and give real-time metrics.
- **Business Impact:** Lower rework and technical debt and reduced maintenance cost, plus higher customer retention, reputation and resource use.

Q4. Define Defect Prevention (DP). Explain the major defect prevention techniques including Root Cause Analysis (RCA) and contrast Defect Prevention with Defect Detection in detail using a comparison table. [10 Marks]

Definition & Overview

Defect Prevention (DP) is a proactive QA strategy that identifies, analyses and removes the root causes of defects throughout the SDLC. Instead of fixing defects after code is written, it stops them being injected, cutting rework cost.

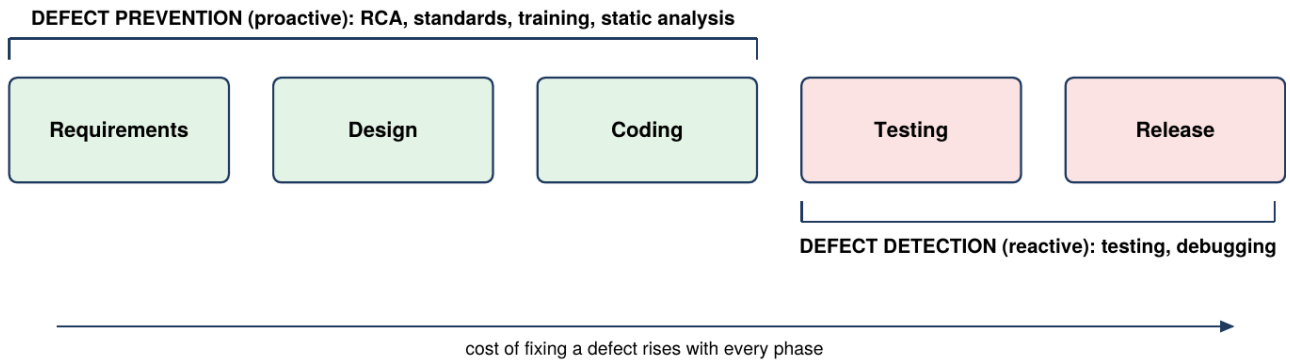


Fig 4: Defect prevention vs defect detection across SDLC phases

- **Concept of Defect Prevention:** A structured process that finds common defect patterns and removes systemic causes such as process weaknesses, ambiguous requirements and communication gaps before bugs appear.

- **Technique 1: Root Cause Analysis (RCA):** Finds the underlying cause of a defect, not its symptoms (e.g., unclear requirements leading to logic errors), so targeted process fixes can be made. Common methods: Fishbone Diagram and 5 Whys.
- **Technique 2: Defect Classification:** Categorises past defects by type (requirements, design, coding, documentation) and severity. This shows which phase injects most bugs and where to put resources and training.
- **Technique 3: Process Improvements:** RCA findings update guidelines, coding standards and templates. Mandatory design reviews, pair programming and formal inspections prevent repeat mistakes.
- **Technique 4: Training and Knowledge Sharing:** Teach engineers about past defect patterns, security risks and domain complexity, using post-mortems and workshops to build capability.
- **Technique 5: Automated Static Tools:** Static analysis in the IDE catches vulnerabilities, syntax violations and memory leaks before execution, enforcing standards and reducing defect leakage into testing.
- **Economic Benefit:** A defect prevented in requirements or design costs far less than one fixed in system testing or maintenance, protecting schedules and reputation.
- **Role in Continuous Improvement:** DP is the operational core of frameworks such as TQM and CMMI; analysing defect trends gives a feedback loop that improves each release.

Attribute	Defect Prevention (DP)	Defect Detection (DD)
Approach	Proactive: avoid defect injection before it occurs.	Reactive: find existing bugs after code execution.
SDLC Stage	Continuously in requirements, design and planning.	During testing, execution and post-build evaluation.
Core Goal	Remove systemic root causes and improve processes.	Identify, isolate and log bugs for developers to fix.
Cost and Effort	Lower cost, as it avoids expensive rework.	High rework cost from late fixes and re-testing.
Primary Tools	RCA (Fishbone, 5 Whys), static analysis, coding standards.	Unit and integration testing, manual inspections, debuggers.

- **Strategic Integration:** An effective SQA strategy combines both, with the main emphasis on prevention. Detection acts as a safety net for residual defects, together giving high reliability and customer satisfaction.

Q5. Describe the complete step-by-step methodology of Software Quality Metrics Analysis and explain how statistical and graphical tools (Control Charts, Pareto Charts, Scatter Diagrams, and Trend Lines) drive continuous quality improvement. [10 Marks]

Definition & Overview

Software Quality Metrics Analysis systematically gathers, validates, interprets and acts on quantitative software data. Statistical and graphical techniques turn raw defect and performance data into insights that stabilise processes and drive continuous improvement.

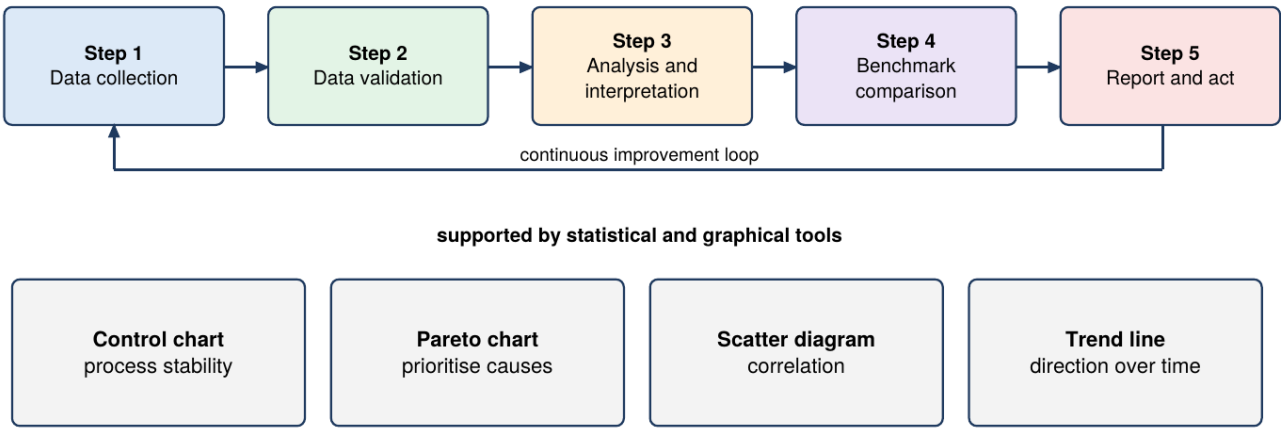


Fig 5: Metrics analysis methodology and supporting tools

- **Step 1: Data Collection:** Collect defect counts, lines of code, review hours and test results from defect trackers, inspection logs, test suites and project tools, using standard protocols for completeness.
- **Step 2: Data Validation:** Check for inaccuracies, duplicates, missing values and anomalies against organisational measurement definitions, since bad data leads to bad decisions.
- **Step 3: Analysis and Interpretation:** Apply statistics to find trends and variations and compute product, process and project metrics such as defect density and DRE.
- **Step 4: Comparison with Benchmarks:** Compare results with baselines, previous releases or industry benchmarks to reveal capability gaps, e.g., defect density against the last release.
- **Step 5: Reporting and Action:** Share findings through dashboards and reports, then plan and execute corrective actions, closing the improvement loop for future SDLC cycles.

- **Control Charts:** Plot a metric against a Central Line (CL), Upper Control Limit (UCL) and Lower Control Limit (LCL) to separate random variation from abnormal variation. Points outside the limits signal instability needing investigation.
- **Pareto Charts:** Rank causes by frequency with a cumulative percentage line, showing that about 80% of defects come from 20% of modules or causes, so testing effort goes to critical sources.
- **Scatter Diagrams:** Plot paired data to find correlation, e.g., cyclomatic complexity vs defect count shows whether complex code causes more bugs and guides refactoring.
- **Trend Lines:** Show direction of change across builds; defect density falling from 1.5 to 0.9 per KLOC confirms a 40% improvement trend.
- **Driving Continuous Improvement:** Used together within the five-step framework, these tools give full visibility so teams can remove root causes and achieve predictable quality.

5-MARK QUESTIONS & ANSWERS

Q6. Explain the PDCA Cycle (Plan-Do-Check-Act / Deming Cycle) as a continuous quality improvement model in software process management. [5 Marks]

Definition & Overview

The PDCA Cycle (Plan-Do-Check-Act), or Deming Cycle, is a four-stage iterative model for continuous quality improvement in TQM, used to solve quality problems and standardise best practice.

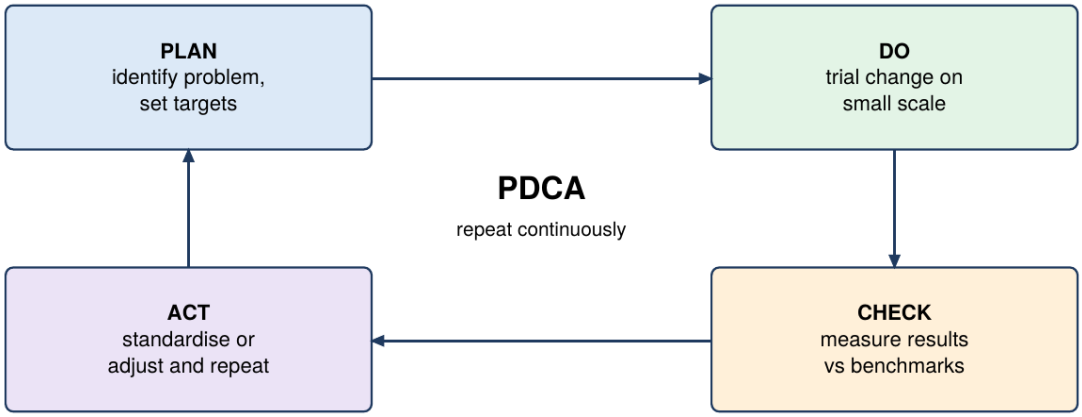


Fig 6: The PDCA (Deming) cycle

- **Plan:** Identify bottlenecks, analyse root causes of defects and design targeted improvements, with clear metrics, benchmarks and strategy.
- **Do:** Implement the change on a small, controlled scale, e.g., trial a static analysis tool or review checklist on one module, to limit risk.
- **Check:** Measure results against the benchmarks using metrics such as review efficiency and defect density to see if the change worked.
- **Act:** If successful, standardise the new process organisation-wide; if not, analyse why, adjust and restart the cycle.
- **Role in SQA:** Embeds continuous learning, preventing process stagnation, reducing defect injection and raising reliability with each iteration.

Q7. What is Benchmarking in SQA? Explain its core objective and describe the three major types of benchmarking used in software development. [5 Marks]

Definition & Overview

Benchmarking compares an organisation's processes, quality metrics or product performance against industry standards or leading organisations to find gaps and adopt proven best practices.

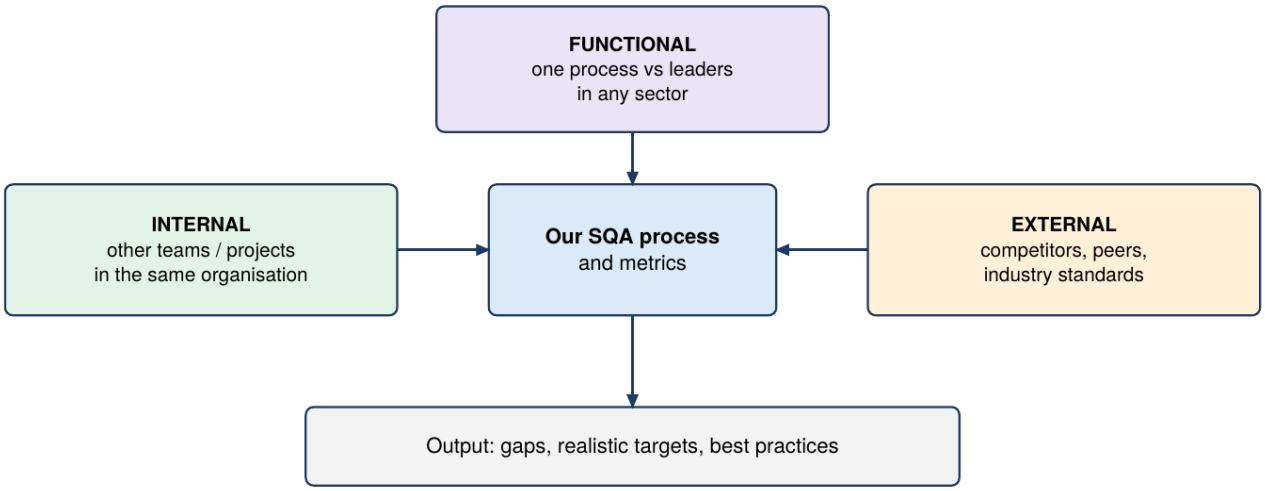


Fig 7: Types of benchmarking

- **Objective:** Set realistic quality targets, expose inefficiencies and drive competitive improvement by learning better defect tracking, testing and release methods from leaders.
- **Internal Benchmarking:** Compares teams, divisions or projects inside the same organisation, finds top internal performers and transfers knowledge. It is cheap because data is easy to access.
- **External Benchmarking:** Compares metrics with peers, competitors or global standards, giving an objective view of market competitiveness and technology gaps.
- **Functional Benchmarking:** Compares a specific function or process (e.g., automated regression testing, release pipeline) with industry leaders regardless of sector, for deep optimisation.
- **Significance in SQA:** Prevents complacency with market-based performance standards and guides SQA planning, resource allocation and modernisation.

Q8. Explain Pareto Analysis (the 80/20 principle) and describe its practical application in software defect management and prioritization. [5 Marks]

Definition & Overview

Pareto Analysis, based on Vilfredo Pareto's 80/20 principle, prioritises quality effort. In SQA, about 80% of defects come from about 20% of causes or code components.

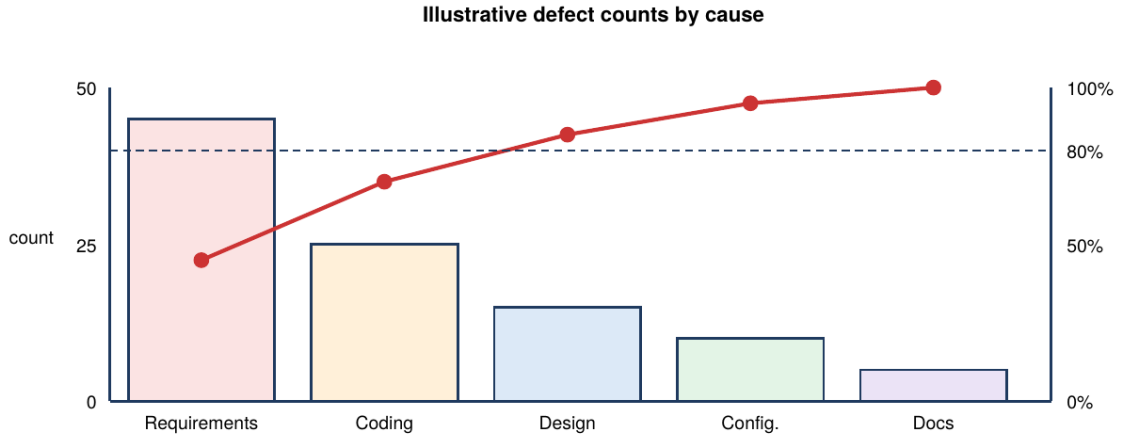


Fig 8: Pareto chart of defect causes (illustrative data)

- **The 80/20 Principle:** A small share of causes produces most problems; defects cluster in a few complex modules or poorly specified requirements rather than spreading evenly.
- **Construction of a Pareto Chart:** Bars in descending order of frequency plus a cumulative percentage line; the left axis shows defect count or rework cost and the right axis cumulative % up to 100.
- **Identifying the Vital Few:** Sorting separates the "vital few" causes from the "trivial many", showing which top 20% of categories create 80% of instability.
- **Application in Defect Prioritisation:** Direct testing, review and refactoring effort at the high-defect modules instead of spreading it evenly across all files.
- **Impact:** Fixing the vital few gives the largest quality gain for the least effort, speeding bug resolution and cutting rework cost.

Q9. Describe Control Charts in software quality control, detailing the Central Line (CL), Upper Control Limit (UCL), Lower Control Limit (LCL), and process variations. [5 Marks]

Definition & Overview

Control Charts are statistical process control tools that track performance over time, showing whether a development process is statistically stable or disrupted.

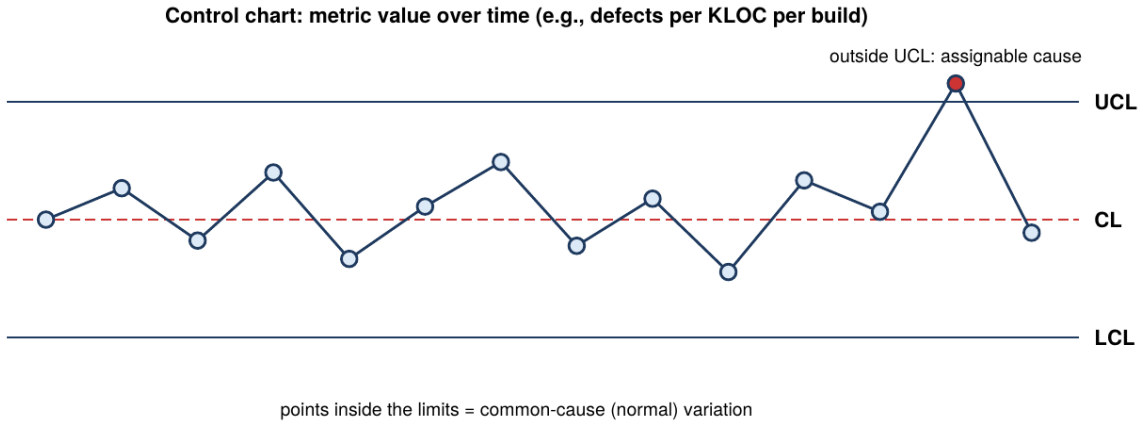


Fig 9: Control chart with CL, UCL and LCL

- **Structure:** Metric values are plotted in time order against three lines: the Central Line (CL, the process average), the Upper Control Limit (UCL) and the Lower Control Limit (LCL), typically set at ±3σ.
- **Normal (Common Cause) Variation:** Random fluctuation inherent in a stable process. Points inside the limits with no systematic pattern mean the process is stable and predictable.
- **Abnormal (Assignable Cause) Variation:** Caused by specific events such as an untrained developer, tool failure or unstable test environment. Points outside the limits or non-random trends signal it.
- **Software Applications:** Monitor defects per KLOC, review efficiency, build success rates and cycle times to catch degradation before release.
- **Management Action:** When a point crosses a limit, pause, run root cause analysis and remove the assignable cause, keeping the process consistent across SDLC phases.

Q10. Explain the Cause-and-Effect (Ishikawa/Fishbone) Diagram and detail the five typical cause categories used in software defect analysis. [5 Marks]

Definition & Overview

The Cause-and-Effect (Ishikawa or Fishbone) Diagram, developed by Kaoru Ishikawa, is a visual RCA tool. The problem (effect) sits at the fish's head, and potential causes branch off the spine as categories and sub-causes.

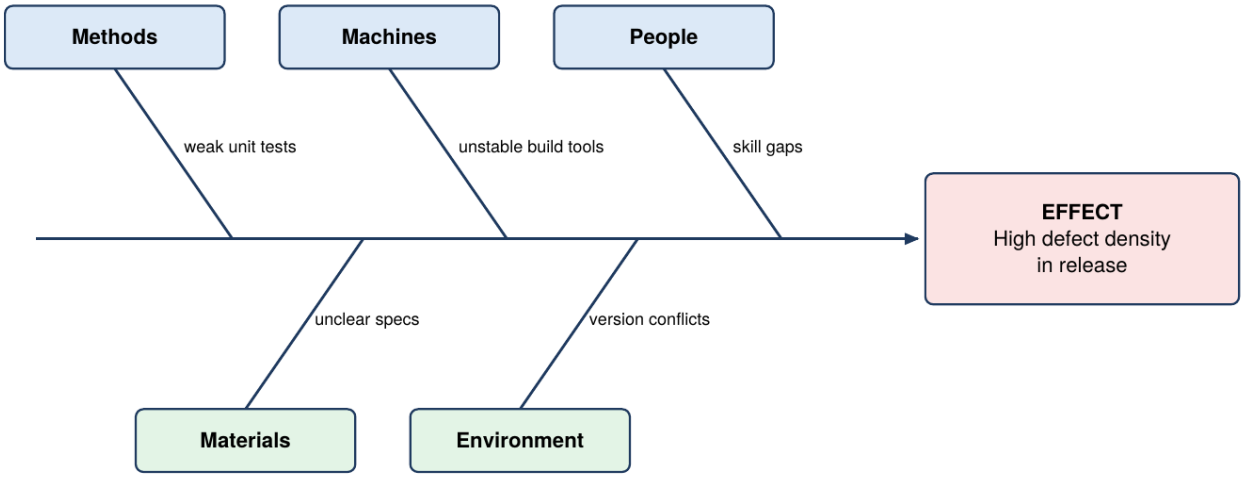


Fig 10: Fishbone diagram for high defect density

- **Methods:** Flaws in processes, design methods, test procedures or coding standards, e.g., weak unit-testing protocols or unreviewed requirements.
- **Machines:** Hardware, workstations, faulty development tools or unstable build and test environments, e.g., server downtime or compiler bugs.
- **People:** Human factors: skill gaps, insufficient training, excessive workload and poor analyst-programmer communication.
- **Materials:** Inputs such as defective third-party libraries, incomplete requirement specifications or corrupted test data.

- **Environment:** Workplace and infrastructure factors such as version-control conflicts or unstable staging servers. Brainstorming across all five categories exposes root causes for prevention.

Q11. Explain Cyclomatic Complexity V(G) and Mean Time to Failure (MTTF) as product quality metrics, providing their mathematical formulas and structural components. [5 Marks]

Definition & Overview

Cyclomatic Complexity and Mean Time to Failure (MTTF) are product quality metrics that measure structural complexity and operational reliability.

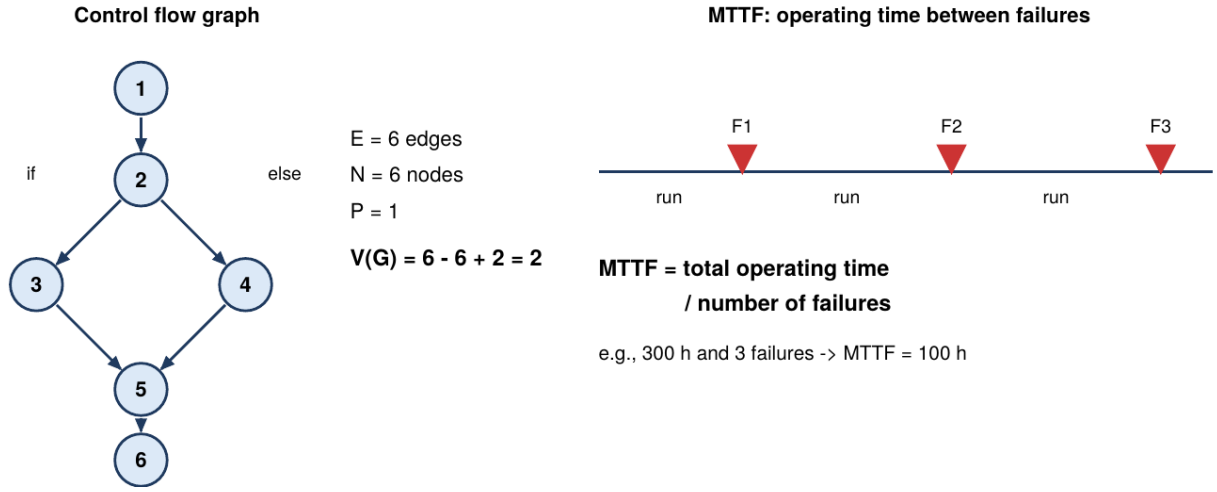


Fig 11: Cyclomatic complexity example and MTTF timeline

- **Cyclomatic Complexity V(G):** Developed by Thomas McCabe; it counts the linearly independent paths through a module's control flow graph, indicating logical difficulty and required testing.
- **Formula:** V(G) = E - N + 2P, where E = edges (control transfers), N = nodes (code blocks), P = connected components (usually 1 for a single program).
- **Significance of V(G):** High values mean many branches, so error-prone, hard-to-maintain code; it gives the minimum number of test cases for full branch coverage.
- **Mean Time to Failure (MTTF):** Average operating time before a failure occurs. A higher MTTF means better reliability.
- **Formula for MTTF:** MTTF = Total operational time / Total number of failures. Tracking it across builds shows whether reliability is growing before deployment.

Q12. Explain Defect Removal Efficiency (DRE) and Rework Percentage as process quality metrics, detailing their mathematical formulas and operational significance. [5 Marks]

Definition & Overview

Defect Removal Efficiency (DRE) and Rework Percentage are process metrics: DRE shows how effective QA activities are, and rework shows how much effort goes to bug fixing.

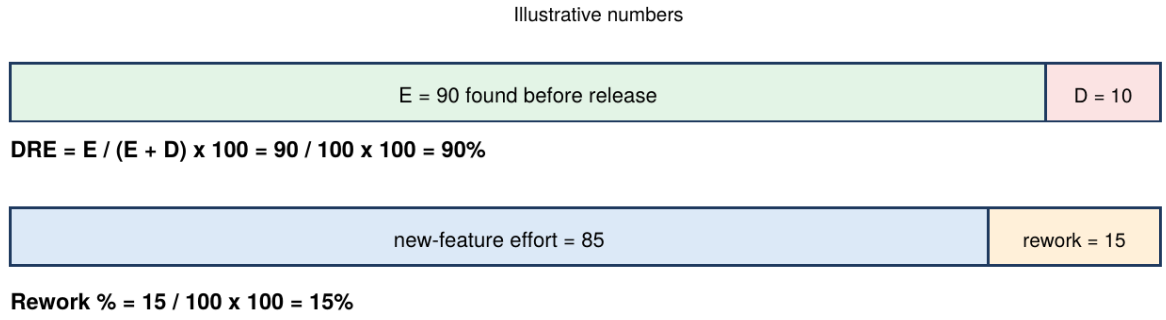


Fig 12: DRE and rework percentage (illustrative numbers)

- **DRE Definition:** The percentage of total defects detected and fixed internally before release; it shows how well verification phases filter defects.
- **DRE Formula:** DRE = (E / (E + D)) x 100, where E = defects found before release (internal testing and reviews) and D = defects reported by customers after release.
- **Significance of DRE:** A value near 100% means nearly all defects are caught before deployment; a low value shows weak testing gates and bug leakage into production.
- **Rework Percentage:** Rework % = (Effort on defect correction / Total project effort) x 100, the share of effort spent fixing defects.

- **Significance of Rework:** High values signal inefficient practices, unclear requirements or low initial code quality; reducing it frees capacity for new features.

Q13. Compare Product Metrics, Process Metrics, and Project Metrics using a structured comparison table across key evaluation dimensions. [5 Marks]

Definition & Overview

Software metrics fall into Product, Process and Project metrics. Each covers a different domain, and together they give a complete framework for quality evaluation and management decisions.

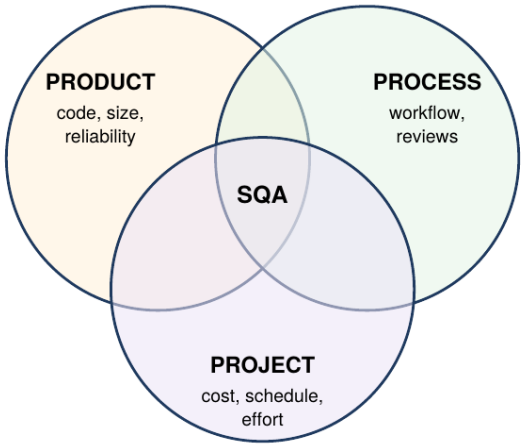


Fig 13: Product, process and project metrics together support SQA

Dimension	Product Metrics	Process Metrics	Project Metrics
Primary Focus	Characteristics of the software product artefact.	Efficiency and quality of development processes.	Project management parameters and resource allocation.
Key Objectives	Assess complexity, size, usability and reliability.	Improve SDLC workflows, raise DRE, reduce rework.	Ensure on-time delivery, budget compliance, optimal resource use.
Typical Examples	LOC, V(G), defect density, MTTF.	DRE, rework %, inspection yield.	Effort variance, schedule variance, CPI.
Primary Users	Developers, architects, QA testers.	Process engineers, SEPG, SQA leads.	Project managers, scrum masters, executives.
Time Horizon	During design, coding and execution.	Continuously across all SDLC phases.	At milestones and completion gates.

2-MARK QUESTIONS & ANSWERS

Q14. Define a Software Quality Metric and state its primary role in decision-making. [2 Marks]

A Software Quality Metric is a quantitative indicator that measures a specific attribute of a software product, process or project.

- **Definition:** It gives an objective numerical measurement of a software attribute, turning qualitative judgement into measurable data.
- **Role in Decision-Making:** It gives managers and engineers an objective baseline for fact-based decisions on product quality, process optimisation and release readiness.

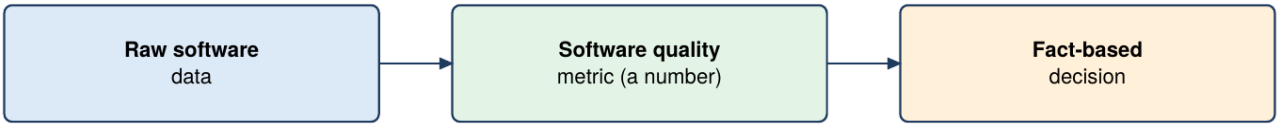


Fig 14: From data to decision

Q15. Define Defect Density (DD) and write its mathematical formula. [2 Marks]

Defect Density measures software quality as the number of defects relative to code size.

Defect Density (DD) = Total Number of Defects / KLOC (thousand lines of code)

- **Definition:** Total confirmed defects found in a module divided by the size of that module.
- **Interpretation:** Lower defect density means higher code quality.

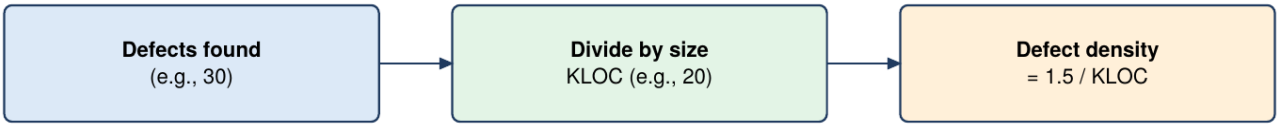


Fig 15: Defect density calculation (illustrative)

Q16. Explain Mean Time to Repair (MTTR) in software maintainability. [2 Marks]

Mean Time to Repair (MTTR) is a maintainability metric showing how quickly defects are fixed.

- **Definition:** The average time the team needs to diagnose, fix, test and deploy a solution for a reported defect.
- **Significance:** A lower MTTR shows better maintainability and debugging, and less downtime during failure recovery.

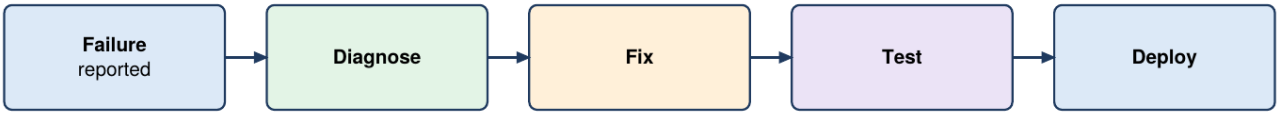


Fig 16: Repair activities covered by MTTR

Q17. Define Inspection Yield and state its purpose in software review processes. [2 Marks]

Inspection Yield is a process metric for peer reviews and formal inspections.

- **Definition:** The percentage (effectiveness) of defects that design and code reviews detect before formal testing.
- **Purpose:** A high yield shows reviews catch bugs early, reducing downstream testing cost and rework.

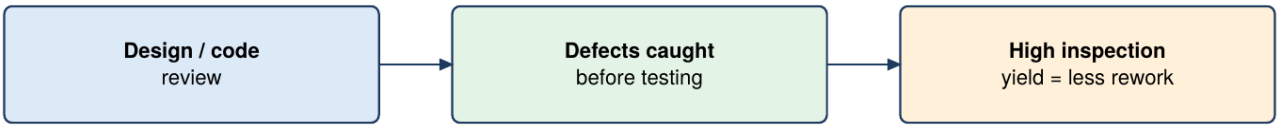
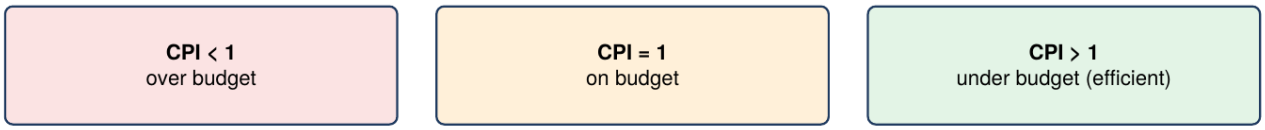


Fig 17: Inspection yield in the review process

Q18. Define Cost Performance Index (CPI) and Effort Variance in project metrics. [2 Marks]

Cost Performance Index (CPI) and Effort Variance are project metrics for efficiency.

- **Effort Variance:** Compares planned and actual effort: Actual Effort - Planned Effort.
- **Cost Performance Index:** CPI = Earned Value / Actual Cost; CPI of 1 or more means budget efficiency. Both warn managers of overruns and misalignment in real time.



CPI = Earned Value / Actual Cost

Effort Variance = Actual effort - Planned effort (positive = overrun)

Fig 18: Reading CPI and effort variance

Q19. Explain the concept of Continuous Improvement (Kaizen) in TQM. [2 Marks]

Continuous Improvement (Kaizen) is a foundational principle of Total Quality Management.

- **Concept:** Focuses on continuous, incremental enhancements to development processes rather than occasional major overhauls.
- **Impact:** Small, disciplined improvements in coding standards, documentation and reviews compound into large quality and productivity gains.

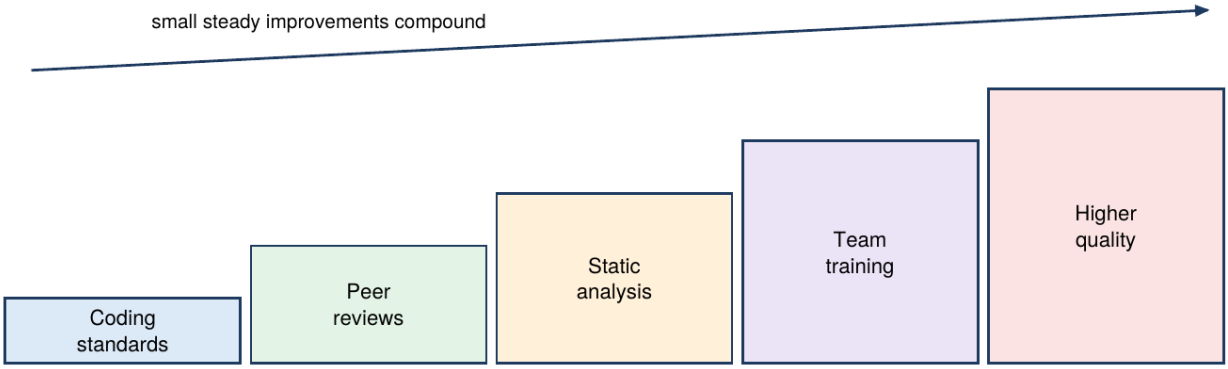


Fig 19: Kaizen: small steps add up

Q20. Briefly describe the 5 Whys Technique used in Root Cause Analysis. [2 Marks]

The 5 Whys Technique is an iterative problem-solving tool used in Root Cause Analysis (RCA).

- **Description:** Repeatedly ask "Why?" (typically five times) to move past surface symptoms until the underlying root cause is found.
- **Example:** For a null pointer failure, successive whys trace the issue from a missing check to unclear requirement specs and communication gaps.

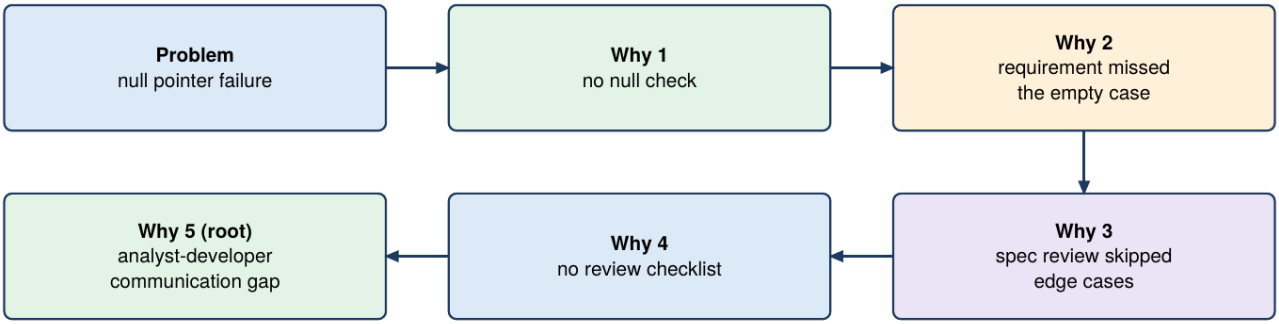


Fig 20: 5 Whys chain (illustrative example)

Q21. What is a Scatter Diagram and how is it used in software metrics analysis? [2 Marks]

A Scatter Diagram is a graphical tool for studying relationships between two variables.

- **Definition:** Plots paired numerical data on two axes to show correlation between two software variables.
- **Software Application:** Compares cyclomatic complexity with defect count to see whether higher complexity causes more bugs.

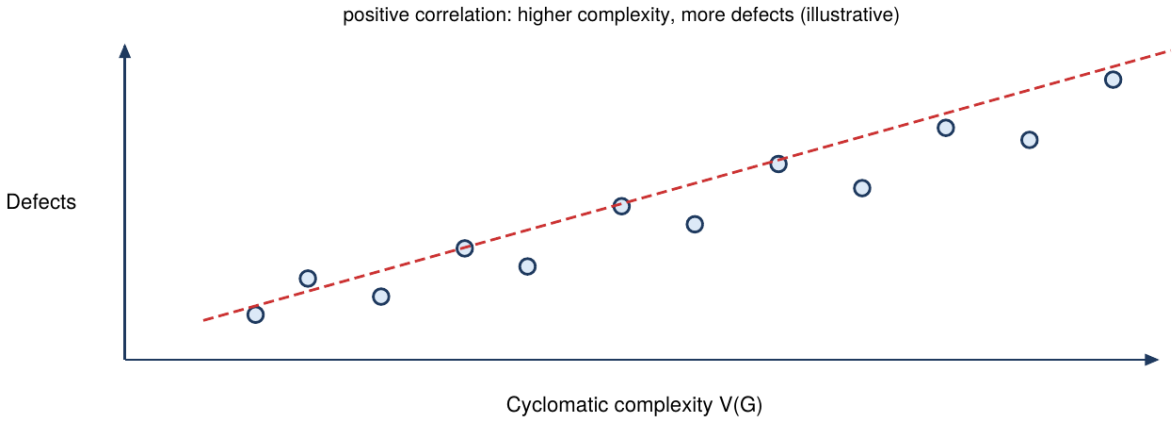


Fig 21: Scatter diagram: complexity vs defects (illustrative)

Q22. If the defect density of a software system drops from 1.5 defects/KLOC in Version 1 to 0.9 defects/KLOC in Version 2, calculate the percentage quality improvement. [2 Marks]

Defect density reduction across releases measures quality improvement.

- **Calculation:** Defect density reduction = 1.5 - 0.9 = 0.6 defects/KLOC.

Percentage Quality Improvement = (0.6 / 1.5) × 100 = 0.40 × 100 = 40%

- **Conclusion:** Version 2 achieved a 40% quality improvement over Version 1.

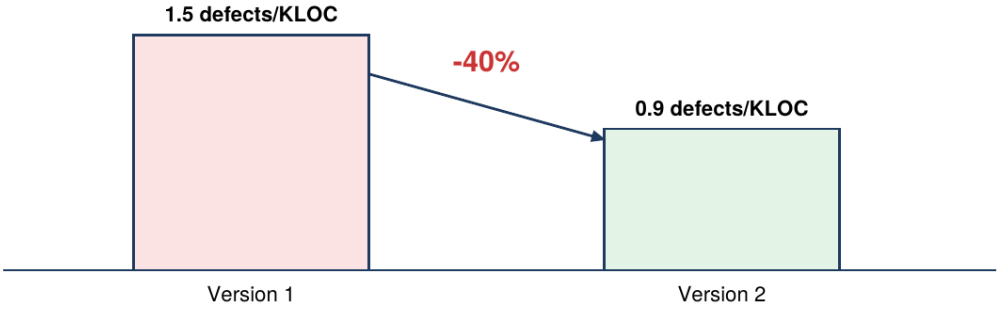


Fig 22: Defect density in Version 1 vs Version 2